

BiteFixes Enterprise Kernel (BEK)

La idea es sencilla:

Todo el sistema gira alrededor de un Kernel.

Igual que Windows tiene un Kernel.

Igual que Linux tiene un Kernel.

BiteFixes también debería tener uno.

¿Qué hace un Kernel?

Nunca muestra pantallas.

Nunca responde WhatsApp.

Nunca sabe si existe Flutter.

Nunca sabe si existe React.

Solo coordina.

Channels

WhatsApp

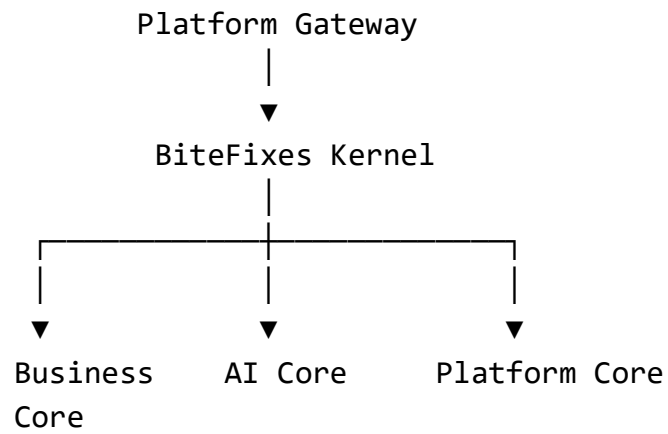
Web

Mobile

REST API

Admin Panel





Dentro del Kernel

El Kernel únicamente coordina.

Kernel



Authentication



Authorization



Company Context



Transaction



Event Bus

↓

Audit

↓

Configuration

↓

Dependency Injection

↓

Module Manager

El Kernel no conoce tickets.

No conoce clientes.

Eso pertenece al Business Core.

El Business Core

Aquí vive el negocio.

Customer Domain

Device Domain

Ticket Domain

Inventory Domain

Knowledge Domain

Service Domain

Billing Domain

Warranty Domain

CRM Domain

Cada dominio es independiente.

El AI Core

Aquí vive Bitey.

Perception

Understanding

Memory

Intent

Planning

Decision

Reflection

Learning

Nunca accede directamente a la base de datos.

Siempre utiliza el Business Core.

El Platform Core

Aquí viven todos los servicios técnicos.

Logging

Monitoring

Storage

Email

WhatsApp

Cache

Queue

Search

Translation

Notifications

Todos reutilizables.

Domain Isolation

Cada dominio debe tener exactamente la misma estructura.

Ejemplo:

Customer

Entity

Repository

Use Cases

Policies

Events

DTO

API

Tests

El dominio Ticket tendría exactamente la misma organización.

Esto hace que el proyecto sea uniforme.

Domain Contracts

Los dominios nunca conocen la implementación de otros.

Solo conocen contratos.

Ejemplo:

Ticket

↓

ICustomerRepository

↓

Customer Domain

Nunca importa directamente otra clase.

Event Driven Domains

Los dominios colaboran mediante eventos.

Ticket Created

↓

Notification

↓

Analytics

↓

Timeline

↓

Bitey

↓

Reports

No existen dependencias fuertes.

Enterprise Timeline

Una de las ideas que considero más valiosas.

No guardar únicamente mensajes.

Guardar absolutamente toda la historia.

Customer



Conversation



Ticket



Estimate



Repair



Warranty



Invoice



Survey

Todo queda relacionado cronológicamente.

Company Context

Toda operación comienza igual.

Request



Authentication



Company



User



Language



Permissions



Business Rules



Execution

Así ningún módulo necesita preguntar constantemente quién es la empresa.

Universal Configuration

Toda configuración centralizada.

Company



Languages



Business Hours



Policies



Workflows



Notifications



AI



Appearance

No debería existir configuración dispersa por el código.

Enterprise Cache

No cachear únicamente consultas SQL.

Cachear conocimiento.

Knowledge

↓

Services

↓

Policies

↓

Languages

↓

Configuration

↓

Statistics

Reducirá mucho el costo computacional.

Search Layer

No hacer consultas directamente.

Todo pasa por un buscador interno.

Search



Customer



Ticket



Device



Knowledge



Conversation

En el futuro podrá cambiarse PostgreSQL por Elasticsearch, OpenSearch o un índice vectorial sin modificar los módulos.

Digital Twin

Aquí está una de las ideas más potentes.

Cada empresa tendrá un gemelo digital.

Company



Customers

↓

Technicians

↓

Tickets

↓

Inventory

↓

Services

↓

Knowledge

↓

History

↓

KPIs

Bitey interactúa con ese modelo.

No con tablas.

Enterprise State Machine

Todas las entidades importantes tienen máquinas de estado.

Ejemplo:

Ticket



Open



Diagnosis



Waiting Parts



Repair



Testing



Ready



Delivered



Closed

Nunca existirán cambios inválidos.

El mayor cambio que introduciría

Además de todo lo anterior, incorporaría un **Domain Event Store**.

En lugar de registrar únicamente el estado final de las entidades, guardaríamos también los eventos de negocio más relevantes:

CustomerRegistered

↓

ConversationStarted

↓

IntentDetected

↓

KnowledgeMatched

↓

TicketCreated

↓

TechnicianAssigned

↓

RepairCompleted

↓

WarrantyActivated

No significa reemplazar el modelo relacional ni adoptar Event Sourcing completo. La base de datos seguirá almacenando el estado actual para que las consultas sean rápidas. El Event Store actuará como un historial estructurado de eventos importantes.

Esto aporta varias ventajas:

- Auditoría detallada de procesos.
- Reconstrucción de la historia de una entidad.
- Alimentación de métricas y analítica.
- Entrenamiento y evaluación futura de Bity.
- Integración sencilla con nuevos módulos sin acoplarlos al núcleo.

El siguiente nivel

Con la arquitectura ya definida, el siguiente trabajo dejaría de ser puramente técnico y pasaría a ser **ingeniería de plataforma**: establecer estándares de calidad, pruebas, observabilidad, seguridad, versionado de APIs, migraciones y despliegue continuo. Esos elementos son los que convierten un proyecto bien programado en una plataforma empresarial preparada para operar de forma fiable durante años.